

Deep Learning Report

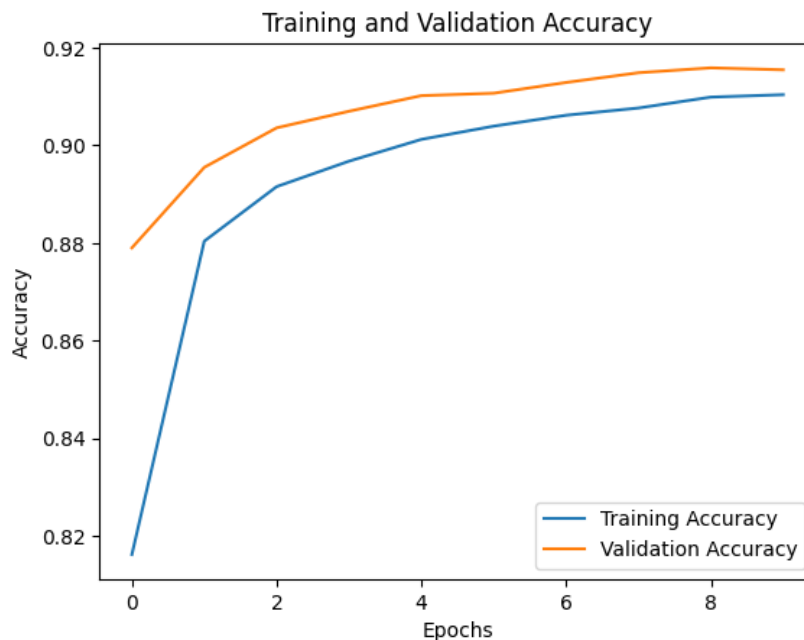
Assignment 1

Muhammad Hassan Khalid

21L-5692

Question 1

I implemented a logistic regression model on MNIST dataset flattened the 28x28 input and passed them through a SoftMax layer. Model trained using Stochastic Gradient Descent optimizer for 10 epochs, accuracy of 91.55% was achieved. Logistic regression, while a linear model, performed reasonably well on this simple classification task due to the well-structured nature of the MNIST dataset. Because it assumes linear decision boundaries more advanced models like neural networks may be needed for more complex data. The model demonstrates the effectiveness of logistic regression as a baseline for classification tasks.



Optimizer Performance Results

SGD (Stochastic Gradient Descent)

It achieved an accuracy of 91.59%. SGD performs well with small datasets when the learning rate is carefully tuned. On the contrary it converges very slowly as it applies the same learning rate to all the parameters. It is not efficient for complex tasks but can work well in shallow networks due to its simplicity. SGD works best when the dataset is not very complex, and training time is not a problem. It works where computational simplicity is preferred.

SGD with Momentum

It achieved an accuracy of 92.41%. SGD with Momentum helps it accelerate in the direction of the minima and avoid the local minima by adding a velocity term. It works best when the gradient oscillates in directions helping to smooth out the learning. SGD with Momentum works best for non-convex cost functions and tasks where training convergence is slow. It reduces oscillations and improves convergence speed.

NAG (Nesterov's Accelerated Gradient)

It achieved an accuracy of 92.32%. It is like momentum-based networks but uses the estimated future gradient to make the optimization more stable. NAG is more cautious and careful while updating the weights. NAG works best when the gradient varies significantly and prediction for future steps for smoother convergence is required. It is more stable for optimization when momentum causes overshooting.

Adagrad (Adaptive Gradient Algorithm)

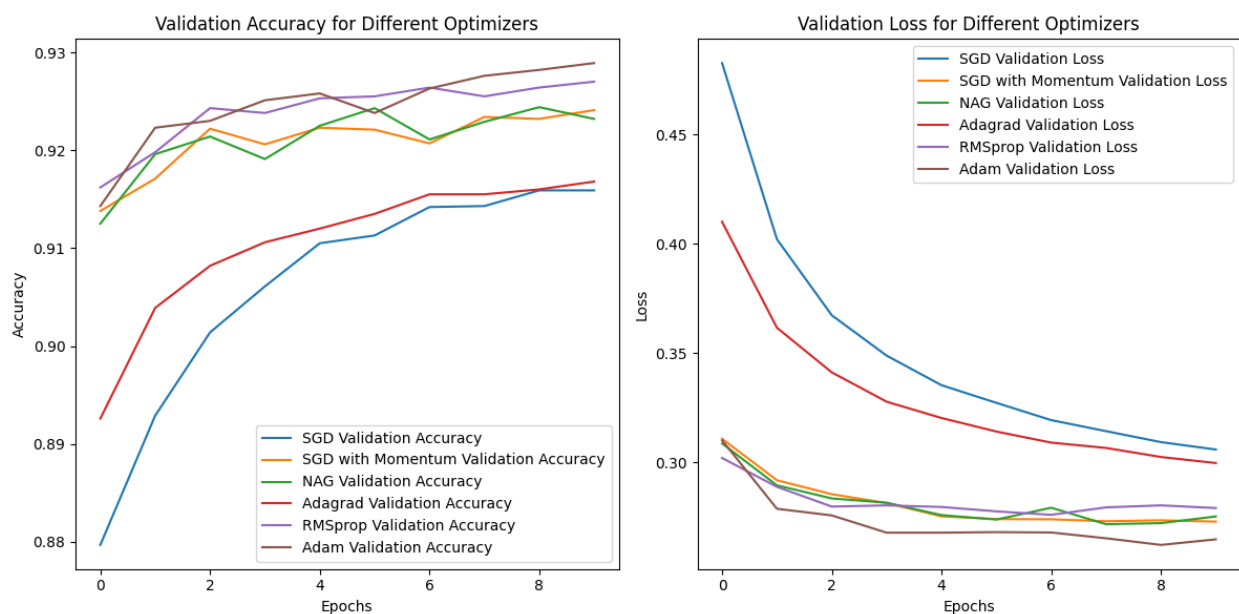
It achieved an accuracy of 91.68%. Adagrad adjusts learning rates based on each parameter's update history, thus making it a good choice for sparse datasets. But its learning rate can decay too fast making it a bad choice for long-term training. The best use for Adagrad is for sparse data and cases where the learning rate needs to vary drastically for different features. But it may not be suitable for large deep networks as learning rate decay limits performance.

RMSprop (Root Mean Square Propagation)

It achieved an accuracy of 92.70%. RMSprop solves Adagrad's learning rate decay issue by maintaining a moving average of squared gradients. Allowing better performance in deeper networks by adapting learning rates more efficiently. It works best for recurrent neural networks and tasks with noisy or non-stationary data. Well-suited for deep learning tasks with long training cycles.

Adam (Adaptive Moment Estimation)

It achieved an accuracy of 92.89%. It combines the benefits of both momentum and RMSprop, by incorporating adaptive learning rate and momentum to improve convergence. It is versatile and performs good in almost all scenarios. Adam works best for most tasks due to its adaptive nature. It performs well in large datasets, deep networks and tasks where training speed and accuracy are important. It is the default optimizer for many deep learning tasks due to its robustness.



Adam performed the best overall due to its balance of adaptive learning rates and momentum, making it robust across a wide range of tasks. SGD with Momentum and RMSprop also showed strong performance, RMSprop showed a slightly better performance due to its ability to adapt the learning rate

dynamically. Adagrad works well for sparse data but isn't as efficient for more complex tasks due to learning rate decay.

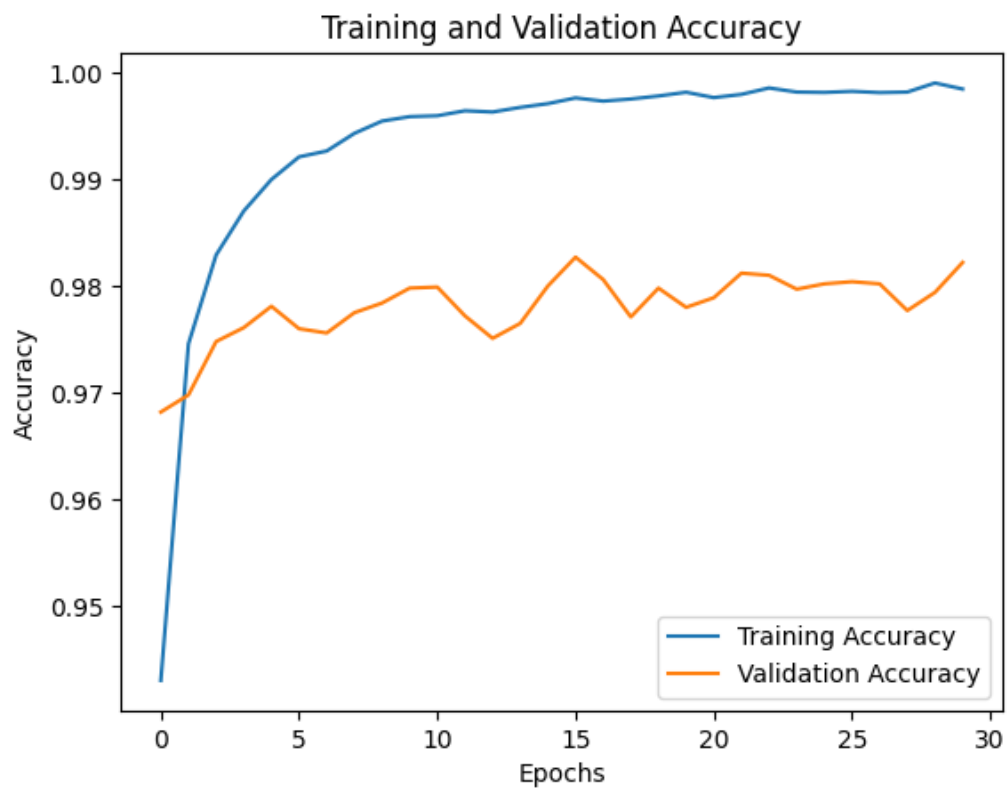
Hyperparameter Tuning using KerasTuner

The KerasTuner library was used to perform the hyperparameter tuning on a logistic regression model for the MNIST dataset. The process focused on optimizing key parameters such as the learning rate, batch size, and number of units in the hidden layer, with the Adam optimizer selected for training due to its adaptive learning rate, providing faster convergence in training deep learning models. RandomSearch method was chosen to explore different combinations of these hyperparameters. Focus was on optimizing the following parameters:

- **Learning rate** (0.01, 0.001, 0.0001)
- **Batch size** (16, 32, 64)
- **Number of units in the hidden layer** (32 to 512, with a step of 32)

The best hyperparameters achieved a accuracy of 98.23%, the optimal hyperparameters were found to be a learning rate of 0.001, batch size of 16, and 320 units in the hidden layer.

The tuning process significantly improved the model performance then the default model which had lower accuracy and slower convergence. By tuning the learning rate, we were able to balance between rapid learning and stability. Optimizing the batch size helped in improving convergence efficiency. Furthermore, the early stopping helped the model not overfit as the validation loss became constant after 12 epochs. The above result shows the power of hyperparameter tuning in improving model accuracy and efficiency, particularly when using an adaptive optimizer like Adam. Adam optimizer performed well because of its combination of momentum and adaptive learning rates, which allowed the model to converge faster and more effectively than with other optimizers.



```
Trial 10 Complete [00h 02m 13s]
val_accuracy: 0.9663000106811523

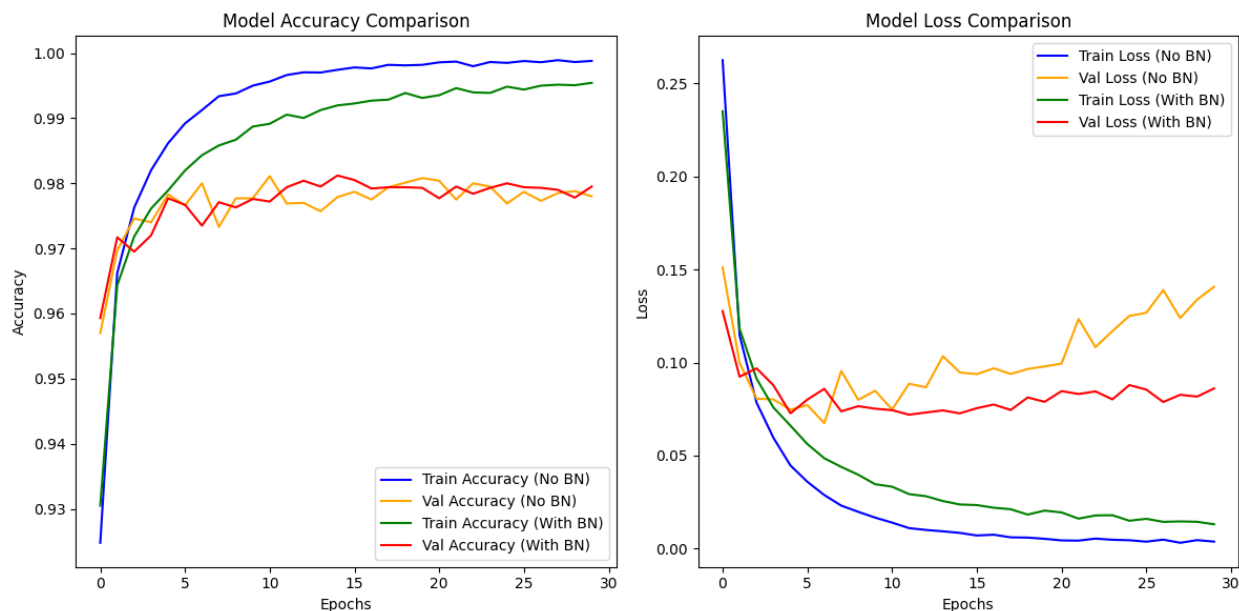
Best val_accuracy So Far: 0.9822999835014343
Total elapsed time: 00h 30m 31s
Best hyperparameters: Learning rate = 0.001, Batch size = 16, Units = 320

Epoch 1/30
3750/3750 ————— 21s 5ms/step - accuracy: 0.9058 - loss: 0.3250 - val_accuracy: 0.9682 - val_loss: 0.1009
Epoch 2/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9743 - loss: 0.0840 - val_accuracy: 0.9698 - val_loss: 0.0936
Epoch 3/30
3750/3750 ————— 20s 5ms/step - accuracy: 0.9832 - loss: 0.0526 - val_accuracy: 0.9748 - val_loss: 0.0770
Epoch 4/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9884 - loss: 0.0355 - val_accuracy: 0.9761 - val_loss: 0.0838
Epoch 5/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9909 - loss: 0.0278 - val_accuracy: 0.9781 - val_loss: 0.0777
Epoch 6/30
3750/3750 ————— 20s 5ms/step - accuracy: 0.9925 - loss: 0.0219 - val_accuracy: 0.9760 - val_loss: 0.0848
Epoch 7/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9933 - loss: 0.0187 - val_accuracy: 0.9756 - val_loss: 0.0993
Epoch 8/30
3750/3750 ————— 22s 5ms/step - accuracy: 0.9947 - loss: 0.0157 - val_accuracy: 0.9775 - val_loss: 0.0904
Epoch 9/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9956 - loss: 0.0127 - val_accuracy: 0.9784 - val_loss: 0.0927
Epoch 10/30
3750/3750 ————— 21s 6ms/step - accuracy: 0.9966 - loss: 0.0105 - val_accuracy: 0.9798 - val_loss: 0.0906
Epoch 11/30
3750/3750 ————— 40s 5ms/step - accuracy: 0.9966 - loss: 0.0094 - val_accuracy: 0.9799 - val_loss: 0.0951
Epoch 12/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9968 - loss: 0.0107 - val_accuracy: 0.9772 - val_loss: 0.1168
Epoch 13/30
3750/3750 ————— 22s 5ms/step - accuracy: 0.9968 - loss: 0.0092 - val_accuracy: 0.9751 - val_loss: 0.1295
Epoch 14/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9976 - loss: 0.0075 - val_accuracy: 0.9765 - val_loss: 0.1467
Epoch 15/30
3750/3750 ————— 23s 6ms/step - accuracy: 0.9969 - loss: 0.0093 - val_accuracy: 0.9800 - val_loss: 0.1121
Epoch 16/30
3750/3750 ————— 40s 6ms/step - accuracy: 0.9977 - loss: 0.0067 - val_accuracy: 0.9827 - val_loss: 0.1044
Epoch 17/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9979 - loss: 0.0065 - val_accuracy: 0.9806 - val_loss: 0.1199
Epoch 18/30
3750/3750 ————— 21s 6ms/step - accuracy: 0.9985 - loss: 0.0051 - val_accuracy: 0.9771 - val_loss: 0.1348
Epoch 19/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9980 - loss: 0.0057 - val_accuracy: 0.9798 - val_loss: 0.1386
Epoch 20/30
3750/3750 ————— 22s 6ms/step - accuracy: 0.9979 - loss: 0.0064 - val_accuracy: 0.9780 - val_loss: 0.1470
```

```
Epoch 21/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9977 - loss: 0.0079 - val_accuracy: 0.9789 - val_loss: 0.1448
Epoch 22/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9982 - loss: 0.0060 - val_accuracy: 0.9812 - val_loss: 0.1429
Epoch 23/30
3750/3750 ————— 21s 5ms/step - accuracy: 0.9988 - loss: 0.0036 - val_accuracy: 0.9810 - val_loss: 0.1406
Epoch 24/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9984 - loss: 0.0048 - val_accuracy: 0.9797 - val_loss: 0.1458
Epoch 25/30
3750/3750 ————— 20s 5ms/step - accuracy: 0.9983 - loss: 0.0063 - val_accuracy: 0.9802 - val_loss: 0.1581
Epoch 26/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9984 - loss: 0.0057 - val_accuracy: 0.9804 - val_loss: 0.1534
Epoch 27/30
3750/3750 ————— 22s 5ms/step - accuracy: 0.9980 - loss: 0.0073 - val_accuracy: 0.9802 - val_loss: 0.1506
Epoch 28/30
3750/3750 ————— 19s 5ms/step - accuracy: 0.9987 - loss: 0.0042 - val_accuracy: 0.9777 - val_loss: 0.1734
Epoch 29/30
3750/3750 ————— 22s 5ms/step - accuracy: 0.9992 - loss: 0.0029 - val_accuracy: 0.9794 - val_loss: 0.1919
Epoch 30/30
3750/3750 ————— 20s 5ms/step - accuracy: 0.9984 - loss: 0.0054 - val_accuracy: 0.9822 - val_loss: 0.1641
313/313 ————— 1s 2ms/step - accuracy: 0.9786 - loss: 0.2051
Tuned model accuracy: 0.9822
```

Question 2

Batch Normalization is a way designed to improve the training of deep neural networks does this by normalizing the inputs to each layer.



Left Plot: Model Accuracy Comparison

Both training accuracies with and without BN reached almost perfect 1 near the end of training. Validation accuracy for both fluctuated but stabilized around 97-98%, and the model using BN showed a more stable trend.

Right Plot: Model Loss Comparison

Both models showed a rapid decrease in training and validation loss initially. The training loss for the model with BN was consistently lower than that of without BN showing better optimization. The validation loss for the model without BN showed an increasing trend towards the end potentially overfitting whereas with BN remained generally lower.

Evaluation of Batch Normalization's Impact

Model Convergence

Batch Normalization improved model convergence. BN model showed a consistent decrease in loss and more stable validation accuracy

throughout the training epochs, meaning that normalization helped the model to learn more effectively by reducing internal covariate shift.

Training Speed

The training time for the model with BN was slightly longer due to the added computations involved in normalizing the inputs. But the overall training speed was enhanced as the model reached a lower loss quickly than without BN model enabling faster convergence.

Generalization

BN played a positive role in the model's generalization capabilities. Validation accuracy for BN model was more stable and did not show signs of overfitting. On the other hand, they were apparent in the model without BN where validation loss increased towards the end of training. This increased stability shows that BN helps reduce overfitting allowing the model to generalize better to the validation data.

The above code experiment affirmed that BN improves the performance of neural networks by improving convergence speed, stability and reducing overfitting. Incorporating BN gives out better performing models. The following were the results

Test accuracy with Batch Normalization: 0.9780, Loss: 0.0965

Test accuracy without Batch Normalization: 0.9807, Loss: 0.1177

Question 3

Gradient Descent is an optimization algorithm that reduces the cost function in various machine learning algorithms.

Batch Gradient Descent (BGD)

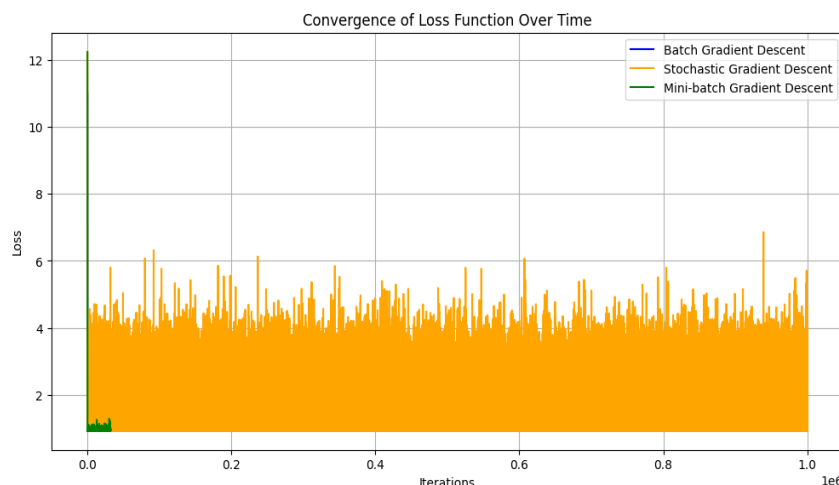
In this application the gradient is computed using the whole dataset. It updates the parameters after computing the gradient over the entire training set. Loss convergence was high initially but gradually decreased over iterations, stabilizing around 0.9327 after 300 iterations.

Stochastic Gradient Descent (SGD)

Instead of using the whole dataset we update the parameters on each individual point introducing more noise into the training process but allows for more frequent updates to the cost function. Initial loss fluctuated significantly, with values above 12, but decreased to around 1.0764 after 500 iterations. However, the loss oscillated more compared to BGD.

Mini-batch Gradient Descent (MBGD)

It is a balance between BGD and SGD by using a subset (mini batch) of the data to compute the gradient reducing noise compared to SGD and allows for faster convergence compared to BGD. Initial loss was slightly lower than in SGD, and it showed more stable convergence. The loss stabilized around 0.9330 after several iterations, like BGD.



Discussion of Pros and Cons

Variant	Pros	Cons
Batch Gradient Descent	- Stable convergence; smooth loss curve. - Utilizes the entire dataset for updates, leading to more accurate gradient estimation.	- Slow updates; can be computationally expensive for large datasets. - Risk of getting stuck in local minima.
Stochastic Gradient Descent	- Faster updates; can escape local minima due to noise. - Suitable for online learning.	- High variance in loss; may lead to oscillations and not converge smoothly. - Requires careful tuning of the learning rate.
Mini-batch Gradient Descent	- Combines advantages of BGD and SGD. - More stable than SGD while being faster than BGD.	- Requires selection of batch size; too small may lead to high variance, too large may resemble BGD. - Slightly more complex to implement than BGD.

Batch Gradient Descent converged smoothly but was computationally expensive for large datasets. SGD showed high variance in loss but updated the parameters quickly which is an advantage in many situations. Mini-batch Gradient Descent provided a balanced approach taking strengths of both methods while reducing weaknesses.

Question 4

The KerasTuner library was used to find the optimal hyperparameters for a neural network model that classified handwritten digits from the MNIST dataset. The tuning focuses on these hyperparameters:

- Learning Rate
- Number of Layers and Units in Each Layer
- Batch Size
- Dropout Rate (if applicable)

To perform hyperparameter tuning a custom HyperModel class was created. The model has a flatten layer that transforms the images of shape 28x28 into a flat vector of size 784. Tuning is done on the number of hidden layers (1 to 3). For each layer, the number of units can be varied between 32 and 512, chosen in steps of 32. A dropout layer is conditionally added based on the tuning parameter to reduce the risk of overfitting. Finally, the model has a dense layer containing 10 units (for the 10 classes of digits) and uses SoftMax activation function to provide class probabilities.

The model is compiled using the Adam optimizer with a learning rate that is also tuning. The loss function used is categorical cross entropy, appropriate for multi-class classification tasks. RandomSearch is used for the hyperparameter space examination and optimizing the validation accuracy. The tuning process was done using the tuner.search method. A maximum of 30 epochs and early stopping based on validation loss was used. Focus was on optimizing the following parameters:

- **Learning Rate:** [0.01, 0.001, 0.0001]
- **Batch Size:** [16, 32, 64]
- **Number of Layers:** [1, 2, 3]
- **Units per Layer:** [32, 64, 96, 128, 160, 192, 224, 256, 288, 320, 352, 384, 416, 448, 480, 512]
- **Dropout Rate:** [0.0, 0.2, 0.5]

The best hyperparameters achieved a accuracy of 98.39%, the optimal hyperparameters were found to be a learning rate of 0.0001, batch size of 16, and [320,96] units in the hidden layer.

The model was then retrained using the best hyperparameters. The training showed a gradual improvement in accuracy:

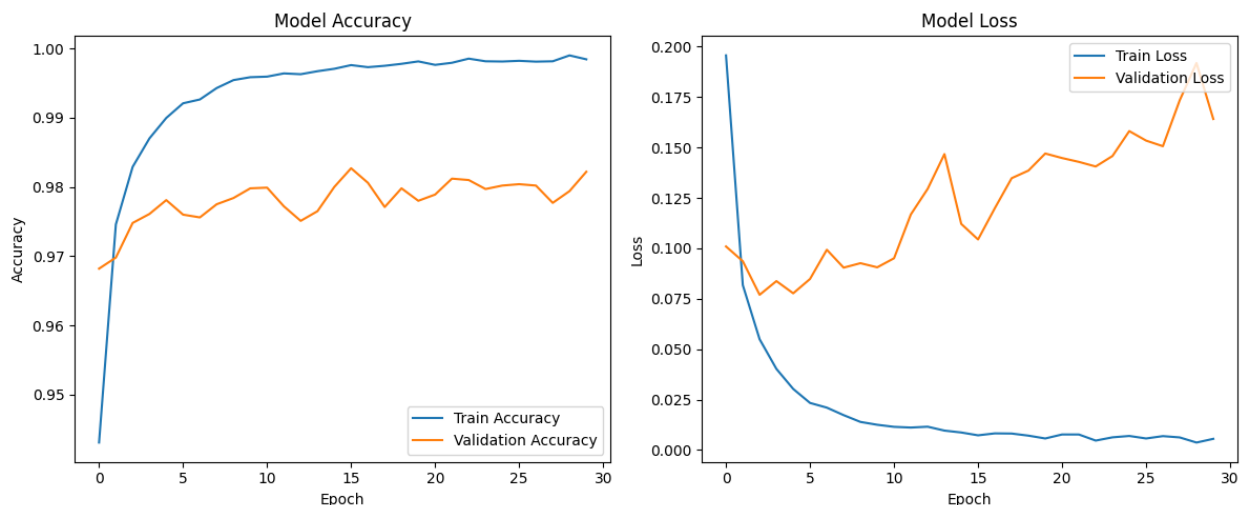
Epoch 1: Training Accuracy: 0.7604, Validation Accuracy: 0.9406

Final Epoch (30): Training Accuracy: 0.9962, Validation Accuracy: 0.9837

The tuned model achieved a test accuracy of 0.9813 when evaluated on the test set.

The smaller learning rate allowed for stable and gradual convergence, reducing risk of overshooting the minima. A batch size of 16 proved to be a balance point for efficient training and performance. The optimal configuration of two layers (320 and 96 units) resulted in a good model complexity and reduces risk of overfitting. Dropout layers were good in preventing overfitting, especially in deeper networks.

The code demonstration affirmed that hyperparameter tuning process for a neural network model does increase its performance. The code implementation is adaptable for various datasets and tasks showing its utility in enhancing neural network performance through effective hyperparameter tuning.



```
Trial 10 Complete [00h 13m 38s]
val_accuracy: 0.9815000295639038

Best val_accuracy So Far: 0.9839000105857849
Total elapsed time: 00h 50m 47s
Best hyperparameters: Learning rate = 0.0001, Batch size = 16, Units = [320, 96]
Epoch 1/30
3750/3750 — 24s 6ms/step - accuracy: 0.7604 - loss: 0.8151 - val_accuracy: 0.9406 - val_loss: 0.2032
Epoch 2/30
3750/3750 — 21s 6ms/step - accuracy: 0.9336 - loss: 0.2309 - val_accuracy: 0.9585 - val_loss: 0.1410
Epoch 3/30
3750/3750 — 42s 6ms/step - accuracy: 0.9515 - loss: 0.1660 - val_accuracy: 0.9665 - val_loss: 0.1103
Epoch 4/30
3750/3750 — 41s 6ms/step - accuracy: 0.9620 - loss: 0.1292 - val_accuracy: 0.9706 - val_loss: 0.0944
Epoch 5/30
3750/3750 — 41s 6ms/step - accuracy: 0.9682 - loss: 0.1048 - val_accuracy: 0.9721 - val_loss: 0.0853
Epoch 6/30
3750/3750 — 41s 6ms/step - accuracy: 0.9730 - loss: 0.0903 - val_accuracy: 0.9762 - val_loss: 0.0762
Epoch 7/30
3750/3750 — 24s 6ms/step - accuracy: 0.9770 - loss: 0.0764 - val_accuracy: 0.9776 - val_loss: 0.0706
Epoch 8/30
3750/3750 — 40s 6ms/step - accuracy: 0.9793 - loss: 0.0682 - val_accuracy: 0.9783 - val_loss: 0.0690
Epoch 9/30
3750/3750 — 22s 6ms/step - accuracy: 0.9821 - loss: 0.0588 - val_accuracy: 0.9788 - val_loss: 0.0688
Epoch 10/30
3750/3750 — 23s 6ms/step - accuracy: 0.9843 - loss: 0.0523 - val_accuracy: 0.9792 - val_loss: 0.0647
Epoch 11/30
3750/3750 — 41s 6ms/step - accuracy: 0.9853 - loss: 0.0467 - val_accuracy: 0.9806 - val_loss: 0.0649
Epoch 12/30
3750/3750 — 24s 6ms/step - accuracy: 0.9875 - loss: 0.0423 - val_accuracy: 0.9821 - val_loss: 0.0605
Epoch 13/30
3750/3750 — 21s 6ms/step - accuracy: 0.9890 - loss: 0.0353 - val_accuracy: 0.9820 - val_loss: 0.0626
Epoch 14/30
3750/3750 — 42s 6ms/step - accuracy: 0.9888 - loss: 0.0349 - val_accuracy: 0.9805 - val_loss: 0.0638
Epoch 15/30
3750/3750 — 21s 6ms/step - accuracy: 0.9907 - loss: 0.0300 - val_accuracy: 0.9820 - val_loss: 0.0603
Epoch 16/30
3750/3750 — 24s 6ms/step - accuracy: 0.9907 - loss: 0.0297 - val_accuracy: 0.9818 - val_loss: 0.0613
Epoch 17/30
3750/3750 — 23s 6ms/step - accuracy: 0.9918 - loss: 0.0269 - val_accuracy: 0.9824 - val_loss: 0.0591
Epoch 18/30
3750/3750 — 21s 6ms/step - accuracy: 0.9927 - loss: 0.0251 - val_accuracy: 0.9834 - val_loss: 0.0601
Epoch 19/30
3750/3750 — 22s 6ms/step - accuracy: 0.9927 - loss: 0.0228 - val_accuracy: 0.9819 - val_loss: 0.0606
```

```
Epoch 10/30
3750/3750 — 23s 6ms/step - accuracy: 0.9843 - loss: 0.0523 - val_accuracy: 0.9792 - val_loss: 0.0647
Epoch 11/30
3750/3750 — 41s 6ms/step - accuracy: 0.9853 - loss: 0.0467 - val_accuracy: 0.9806 - val_loss: 0.0649
Epoch 12/30
3750/3750 — 24s 6ms/step - accuracy: 0.9875 - loss: 0.0423 - val_accuracy: 0.9821 - val_loss: 0.0605
Epoch 13/30
3750/3750 — 21s 6ms/step - accuracy: 0.9890 - loss: 0.0353 - val_accuracy: 0.9820 - val_loss: 0.0626
Epoch 14/30
3750/3750 — 42s 6ms/step - accuracy: 0.9888 - loss: 0.0349 - val_accuracy: 0.9805 - val_loss: 0.0638
Epoch 15/30
3750/3750 — 21s 6ms/step - accuracy: 0.9907 - loss: 0.0300 - val_accuracy: 0.9820 - val_loss: 0.0603
Epoch 16/30
3750/3750 — 24s 6ms/step - accuracy: 0.9907 - loss: 0.0297 - val_accuracy: 0.9818 - val_loss: 0.0613
Epoch 17/30
3750/3750 — 23s 6ms/step - accuracy: 0.9918 - loss: 0.0269 - val_accuracy: 0.9824 - val_loss: 0.0591
Epoch 18/30
3750/3750 — 21s 6ms/step - accuracy: 0.9927 - loss: 0.0251 - val_accuracy: 0.9834 - val_loss: 0.0601
Epoch 19/30
3750/3750 — 22s 6ms/step - accuracy: 0.9927 - loss: 0.0228 - val_accuracy: 0.9819 - val_loss: 0.0606
Epoch 20/30
3750/3750 — 21s 6ms/step - accuracy: 0.9938 - loss: 0.0200 - val_accuracy: 0.9825 - val_loss: 0.0606
Epoch 21/30
3750/3750 — 42s 6ms/step - accuracy: 0.9939 - loss: 0.0198 - val_accuracy: 0.9835 - val_loss: 0.0589
Epoch 22/30
3750/3750 — 42s 6ms/step - accuracy: 0.9945 - loss: 0.0174 - val_accuracy: 0.9834 - val_loss: 0.0624
Epoch 23/30
3750/3750 — 41s 6ms/step - accuracy: 0.9950 - loss: 0.0163 - val_accuracy: 0.9837 - val_loss: 0.0591
Epoch 24/30
3750/3750 — 40s 6ms/step - accuracy: 0.9950 - loss: 0.0157 - val_accuracy: 0.9838 - val_loss: 0.0599
Epoch 25/30
3750/3750 — 23s 6ms/step - accuracy: 0.9951 - loss: 0.0152 - val_accuracy: 0.9837 - val_loss: 0.0648
Epoch 26/30
3750/3750 — 21s 6ms/step - accuracy: 0.9956 - loss: 0.0138 - val_accuracy: 0.9849 - val_loss: 0.0621
Epoch 27/30
3750/3750 — 24s 6ms/step - accuracy: 0.9948 - loss: 0.0162 - val_accuracy: 0.9838 - val_loss: 0.0639
Epoch 28/30
3750/3750 — 40s 6ms/step - accuracy: 0.9966 - loss: 0.0111 - val_accuracy: 0.9826 - val_loss: 0.0650
Epoch 29/30
3750/3750 — 40s 6ms/step - accuracy: 0.9954 - loss: 0.0138 - val_accuracy: 0.9846 - val_loss: 0.0641
Epoch 30/30
3750/3750 — 40s 6ms/step - accuracy: 0.9962 - loss: 0.0106 - val_accuracy: 0.9837 - val_loss: 0.0652
313/313 — 1s 2ms/step - accuracy: 0.9813 - loss: 0.0778
Tuned model accuracy: 0.9837
```